

- **Master assigns dynamic address:** 7-bit address + parity bit (to detect errors; parity = 1 if the number of 1's is even, and parity = 0 if the number of 1's is odd). The device accepts the address and stores it in the hardware register inside it (DADDR).
- **Device ACKs and exits ENTDA:** The device does not participate again in DAA and uses the dynamic address from now on.
- **ENTDA continues for remaining devices:** The same process continues until no devices respond anymore.
- **ENTDA ends:** When the master sees no response during the ENTDA phase, it knows that all devices have been assigned addresses.

In-Band Interrupts — Devices can call for help

In I2C, if a sensor has a problem, it cannot easily alert the master. It needs extra wires. In I3C, devices can send alerts directly on the same bus using In-Band Interrupts (IBI).

For example, if temperature suddenly rises in a server, the sensor sends IBI to BMC, which reacts immediately (increase fan, shutdown, alert admin). This is done without using extra wires, which is very useful in dense server boards.

Where I2C and I3C are being used today

Bus type	Common applications
I2C	EEPROM storage, temperature sensors, RTCs, fan controllers, basic power monitors, legacy server BMC communication
I3C	Sensor hubs in mobile devices, camera modules, SoC integration, server BMC-sensor communication (next-gen IPMI replacements), high-speed telemetry systems

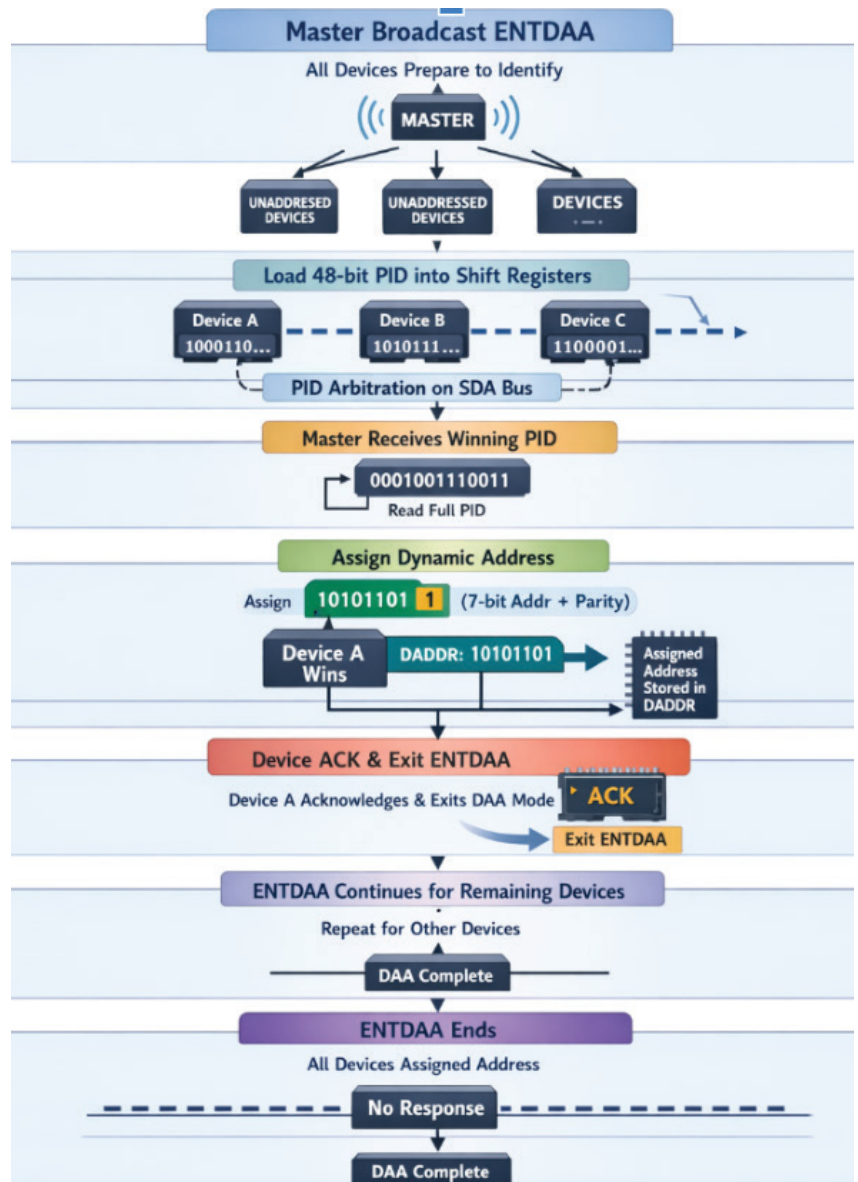


Figure 7: DAA flow in I3C

I2C became widely adopted because of its simplicity, low hardware cost, and reliability. It remains suitable for low-speed and low-complexity peripherals. However, modern embedded systems demand faster communication, lower power consumption, and better scalability.

I3C preserves the simplicity of two-wire communication while introducing features such as dynamic address assignment, in-band interrupts, and push-pull signalling, making it suitable

for today's high-performance sensor-rich systems.

Seen together, I2C and I3C show how good standards grow over time. We don't throw away what already works; we build on it and extend it so it can handle the demands of newer, more advanced hardware. **END** 🐧

By: Priyadharshini S.

The author is an open source enthusiast working in an MNC.